

8086 Assembly Language

Personal learning notes and practice reference

1. Basic Program Structure

These directives define the basic structure of a MASM/TASM-style 8086 program.

```
.model small  
.stack 100h  
.code
```

.model small

Defines the memory model. The small model uses one code segment and one data segment, with each segment able to address up to 64 KB.

.stack 100h

Allocates stack memory. 100h = 256 bytes. The stack is used by instructions such as PUSH, POP, CALL and RET.

.code

Marks the beginning of the code section where executable instructions are written.

2. Registers

Registers are small, fast storage locations inside the CPU.

Register	Name	Common use
AX	Accumulator	Arithmetic, input/output
BX	Base	Base addressing
CX	Counter	Loops
DX	Data	Input/output, data

```
AX → AH + AL  
BX → BH + BL  
CX → CH + CL  
DX → DH + DL
```

3. MOV Instruction

MOV copies data from a source to a destination. Syntax: mov destination, source.

```
mov ax, 10  
mov bx, ax
```

After execution, AX = 10 and BX = 10. The original value in AX is not removed.

4. Operand Types

Immediate: a constant written directly in the instruction.

```
mov ax, 10  
mov ax, 3fh
```

Register: data is taken from another CPU register.

```
mov ax, bx
```

Memory: brackets indicate a memory access.

```
mov ax, [1234h]
```

5. INT 21h

INT 21h provides DOS services. The value in AH selects the service.

```
mov ah, 1  
int 21h
```

Function 01h reads one keyboard character and returns the character in AL.

```
mov ah, 4ch  
int 21h
```

Function 4Ch terminates the program.

6. Taking Character Input

```
mov ah, 1  
int 21h  
mov bl, al
```

AH = 01h selects keyboard input. INT 21h waits for a character. The typed character is returned in AL, then copied into BL.

7. Displaying a Character

```
mov ah, 2  
mov dl, 'A'  
int 21h
```

AH = 02h selects the display-character service. DL contains the character to display.

8. Complete Input/Output Example

```
.model small  
.stack 100h  
.code  
  
main proc  
  
    mov ah, 1  
    int 21h  
  
    mov bl, al  
  
    mov ah, 2  
    mov dl, bl  
    int 21h  
  
    mov ah, 4ch  
    int 21h  
  
main endp  
end main
```

Flow: Keyboard Input → AL → BL → DL → Display Character

9. Arithmetic Instructions

ADD

Adds two values.

```
mov al, 5
add al, 3
```

Result: AL = 8

SUB

Subtracts the source from the destination.

```
mov al, 10
sub al, 3
```

Result: AL = 7

INC

Increases a value by 1.

```
mov ax, 5
inc ax
```

Result: AX = 6

DEC

Decreases a value by 1.

```
mov ax, 5
dec ax
```

Result: AX = 4

10. CMP Instruction

CMP compares two values by internally performing a subtraction, but does not store the result. It updates CPU flags.

```
cmp ax, bx
```

11. JMP Instruction

JMP unconditionally transfers execution to another label.

```
jmp start

start:
    mov ax, 5
    jmp start
```

12. Conditional Jumps

Instruction	Meaning
JE / JZ	Jump if Equal / Zero
JNE / JNZ	Jump if Not Equal / Not Zero
JG	Jump if Greater
JL	Jump if Less
JGE	Jump if Greater or Equal

JLE	Jump if Less or Equal
-----	-----------------------

```
cmp ax, bx
je equal
```

13. Zero Flag (ZF)

ZF indicates whether an operation produces a zero result. ZF = 1 means zero; ZF = 0 means non-zero. Commonly used with JE/JZ and JNE/JNZ.

14. Sign Flag (SF)

SF indicates the sign of a result in signed interpretation. SF = 0 indicates a non-negative result; SF = 1 indicates a negative result.

```
mov al, 5
sub al, 8
```

15. Stack

The stack follows LIFO (Last In, First Out) order.

```
mov ax, 10
push ax

mov ax, 20
pop ax
```

After POP, AX = 10.

16. Procedures

A procedure is a reusable block of code. CALL transfers execution to it and RET returns to the caller.

```
main proc
    ; instructions
main endp
```

17. Comments

Comments are ignored by the assembler and are used to explain code.

```
; This is a comment
mov ah, 1      ; Select keyboard input
int 21h        ; Execute DOS interrupt
```

18. Number Systems

System	Example
Decimal	10
Hexadecimal	0Ah
Binary	1010b

Common suffixes: h = hexadecimal, b = binary.

19. Basic Program Exit

```
mov ah, 4ch
int 21h
```

AH = 4Ch selects the DOS terminate-program service.

20. Learning Roadmap

```
8086 Basics
  ↓
Registers
  ↓
Memory
  ↓
MOV
  ↓
Input / Output
  ↓
Arithmetic
  ↓
Flags
  ↓
CMP
  ↓
Jumps
  ↓
Loops
  ↓
Stack
  ↓
Procedures
  ↓
Reverse Engineering
```

21. Goal

Build a strong foundation in 8086 Assembly Language, low-level programming, and reverse engineering through practical examples and hands-on exercises.